

PART A

Experiment No.05

A.1 Aim: Case study on Basic Programming in PROLOG and Develop a program to implement family tree.

A.2 Prerequisite: Understand knowledge, facts and rules

A.3 Outcome:

After successful completion of this experiment students will be able to

- Use current techniques, tools necessary for computing practice using PROLOG.
- Applying knowledge of computing, applied mathematics to solve engineering problems using logical programming.
- Ability to recognize the need and to engage in lifelong learning for logical programming.

Tools Required: SWI-Prolog

A.4 Theory:

Prolog is a general-purpose logic programming language associated with artificial intelligence and computational linguistics. Prolog has its roots in first-order logic, a formal logic, and unlike many other programming languages, Prolog is declarative: the program logic is expressed in terms of relations, represented as facts and rules. A computation is initiated by running a query over these relations. Prolog is well-suited for specific tasks that benefit from rule-based logical queries such as searching databases, voice control systems, and filling templates.

Rules and facts:

Prolog programs describe relations, defined by means of clauses. Pure Prolog is restricted to Horn clauses. There are two types of clauses: facts and rules. A rule is of the form

Head:- Body.

and is read as "Head is true if Body is true". A rule's body consists of calls to predicates, which are called the rule's goals. The built-in predicate, `/2` (meaning a 2-arity operator with name,) denotes conjunction of goals, and; `/2` denotes disjunction. Conjunctions and disjunctions can only appear in the body, not in the head of a rule.

Clauses with empty bodies are called facts. An example of a fact is:

`cat (tom).` which is equivalent to the rule: `cat(tom) :- true.`

The built-in predicate `true/0` is always true.

Given the above fact, one can ask:

is tom a cat?

?- cat(tom).

Yes

what things are cats?

?- cat(X).

X = tom

Clauses with bodies are called rules. An example of a rule is:

animal(X) :- cat(X).

If we add that rule and ask what things are animals?

?- animal(X).

X = tom

Prolog must be able to handle arithmetic in order to be a useful general purpose programming language. However, arithmetic does not fit nicely into the logical scheme of things. That is, the concept of evaluating an arithmetic expression is in contrast to the straight pattern matching we have seen so far. For this reason, Prolog provides the built-in predicate 'is' that evaluates arithmetic expressions. Its syntax calls for the use of operators.

X is <arithmetic expression>

The variable X is set to the value of the arithmetic expression. On backtracking it is unassigned. The variable X is set to the value of the arithmetic expression. On backtracking it is unassigned. The arithmetic expression looks like an arithmetic expression in any other programming language.

Here is how to use Prolog as a calculator.

?- X is 2 + 2.

X = 4

?- X is 3 * 4 + 2.

X = 14

Parentheses clarify precedence.

?- X is 3 * (4 + 2).

X = 18

?- X is (8 / 4) / 2.

X = 1

In addition to 'is,' Prolog provides a number of operators that compare two numbers. These include 'greater than', 'less than', 'greater or equal than', and 'less or equal than.' They behave more logically, and succeed or fail according to whether the comparison is true or false. Notice the order of the symbols in the greater or equal than and less than or equal operators. They are specifically constructed not to look like an arrow, so that the use arrow symbols in programs is without confusion.

X > Y

X < Y

X >= Y

X <= Y

1. Procedure/ Program:

1 (A). Sample program to demonstrate Rules and facts.

```
weather(phoenix,summer,hot).
weather(la,summer,warm).
weather(phoenix,winter,warm).
```

```
SWI-Prolog -- c:/Users/User/Desktop/waether.pl
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (Multi-threaded, 32 bits, Version 7.2.3)
Copyright (c) 1990-2015 University of Amsterdam, VU Amsterdam
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to redistribute it under certain conditions.
Please visit http://www.swi-prolog.org for details.

For help, use ?- help(Topic). or ?- apropos(Word).

1 ?- weather(City,summer,hot).
City = phoenix.

2 ?- weather(City,_,warm).
City = la.

3 ?- weather(City,winter,warm).
City = phoenix.

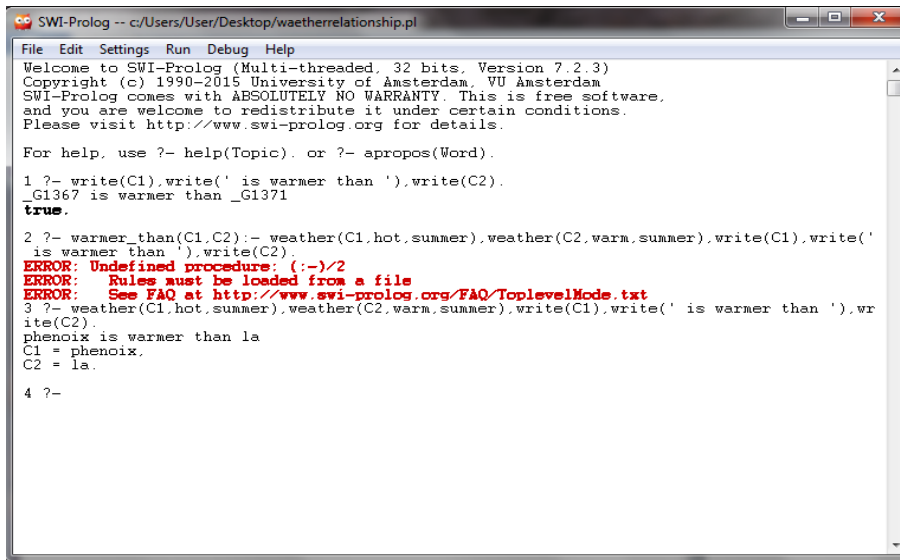
4 ?- weather(la,winter,warm).
false.

5 ?-
```

1.(B) Sample program to demonstrate the relationship in prolog.

```
weather(phoenix,hot,summer).
weather(la,warm,summer).

warmer_than(C1,C2):-
weather(C1,hot,summer),
weather(C2,warm,summer).
```



```

SWI-Prolog -- c:/Users/User/Desktop/waetherrelationship.pl
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (Multi-threaded, 32 bits, Version 7.2.3)
Copyright (c) 1990-2015 University of Amsterdam, VU Amsterdam
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to redistribute it under certain conditions.
Please visit http://www.swi-prolog.org for details.

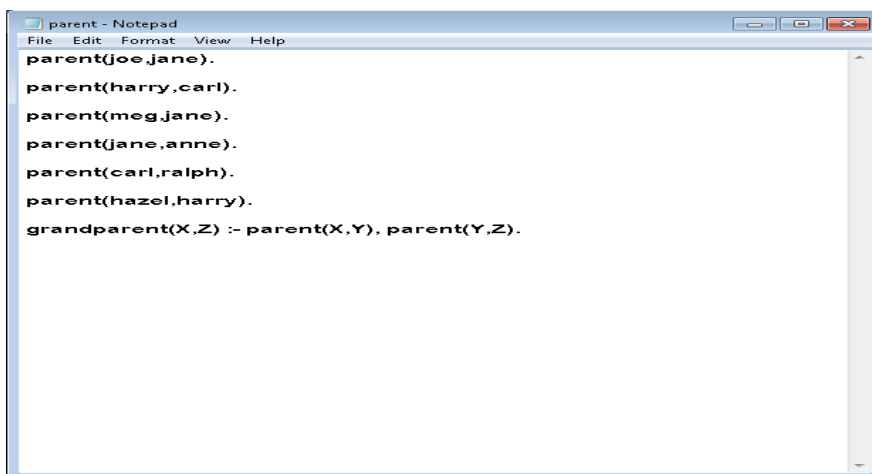
For help, use ?- help(Topic). or ?- apropos(Word).

1 ?- write(C1),write(' is warmer than '),write(C2).
   _G1367 is warmer than _G1371
   true.

2 ?- warmer_than(C1,C2):- weather(C1,hot,summer),weather(C2,warm,summer),write(C1),write('
   is warmer than '),write(C2).
   ERROR: Undefined procedure: (:-)/2
   ERROR: Rules must be loaded from a file
   ERROR: See FAQ at http://www.swi-prolog.org/FAQ/ToplevelMode.txt
3 ?- weather(C1,hot,summer),weather(C2,warm,summer),write(C1),write(' is warmer than '),wr
   ite(C2).
   phoenix is warmer than la
   C1 = phoenix,
   C2 = la.

4 ?-
  
```

1.(C) Sample program to demonstrate the relationship in prolog.



```

parent - Notepad
File Edit Format View Help
parent(joe,jane).
parent(harry,carl).
parent(meg,jane).
parent(jane,anne).
parent(carl,ralph).
parent(hazel,harry).
grandparent(X,Z) :- parent(X,Y), parent(Y,Z).
  
```

```

SWI-Prolog -- c:/Users/USER/Desktop/parent.pl
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (Multi-threaded, 32 bits, Version 7.2.2)
Copyright (c) 1990-2015 University of Amsterdam, VU Amsterdam
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to redistribute it under certain conditions.
Please visit http://www.swi-prolog.org for details.

For help, use ?- help(Topic). or ?- apropos(Word).

1 ?- parent(carl,Y).
Y = ralph.

2 ?- parent(hazel,X).
X = harry.

3 ?- parent(joe,X),parent(meg,X).
X = jane.

4 ?- parent(Y,harry).
Y = hazel.

5 ?- 

```

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.: C-54	Name: Quazi Md Moinuddin
Class: TE Comps C	Batch:C-3
Date of Experiment:07.02.25	Date of Submission:7.02.25
Grade:	

B.1 Software Code written by student:

```

% Facts about family members
parent(john, alice). % John is Alice's parent
parent(john, bob). % John is Bob's parent
parent(mary, alice). % Mary is Alice's parent
parent(mary, bob). % Mary is Bob's parent
parent(bob, charlie). % Bob is Charlie's parent
parent(susan, charlie). % Susan is Charlie's parent

```

```

% Rule to determine siblings (X and Y are siblings if they share at least one
parent)
sibling(X, Y) :-

```

```

    parent(P, X), parent(P, Y), % X and Y share at least one parent P
    X \= Y. % Ensure X is not the same as Y

```

```

% Rule to determine grandparents (GP is a grandparent of X if GP is a parent of
X's parent)

```

```

grandparent(GP, X) :-
    parent(P, X), % P is a parent of X
    parent(GP, P). % GP is a parent of P (X's grandparent)

```

% Rule to determine if someone is an aunt/uncle (AU is an aunt/uncle of X if AU is a sibling of X's parent)

aunt_uncle(AU, X) :-

parent(P, X), % P is a parent of X

sibling(AU, P). % AU is a sibling of P (X's parent)

% Rule to find the children of a particular parent

children(Parent, Child) :-

parent(Parent, Child). % Child is a child of Parent

B.2 Input and Output: \

```
family.pl
File Edit Browse Compile Prolog Pce Help
family.pl
% Facts about family members
parent(john, alice). % John is Alice's parent
parent(john, bob). % John is Bob's parent
parent(mary, alice). % Mary is Alice's parent
parent(mary, bob). % Mary is Bob's parent
parent(bob, charlie). % Bob is Charlie's parent
parent(susan, charlie). % Susan is Charlie's parent

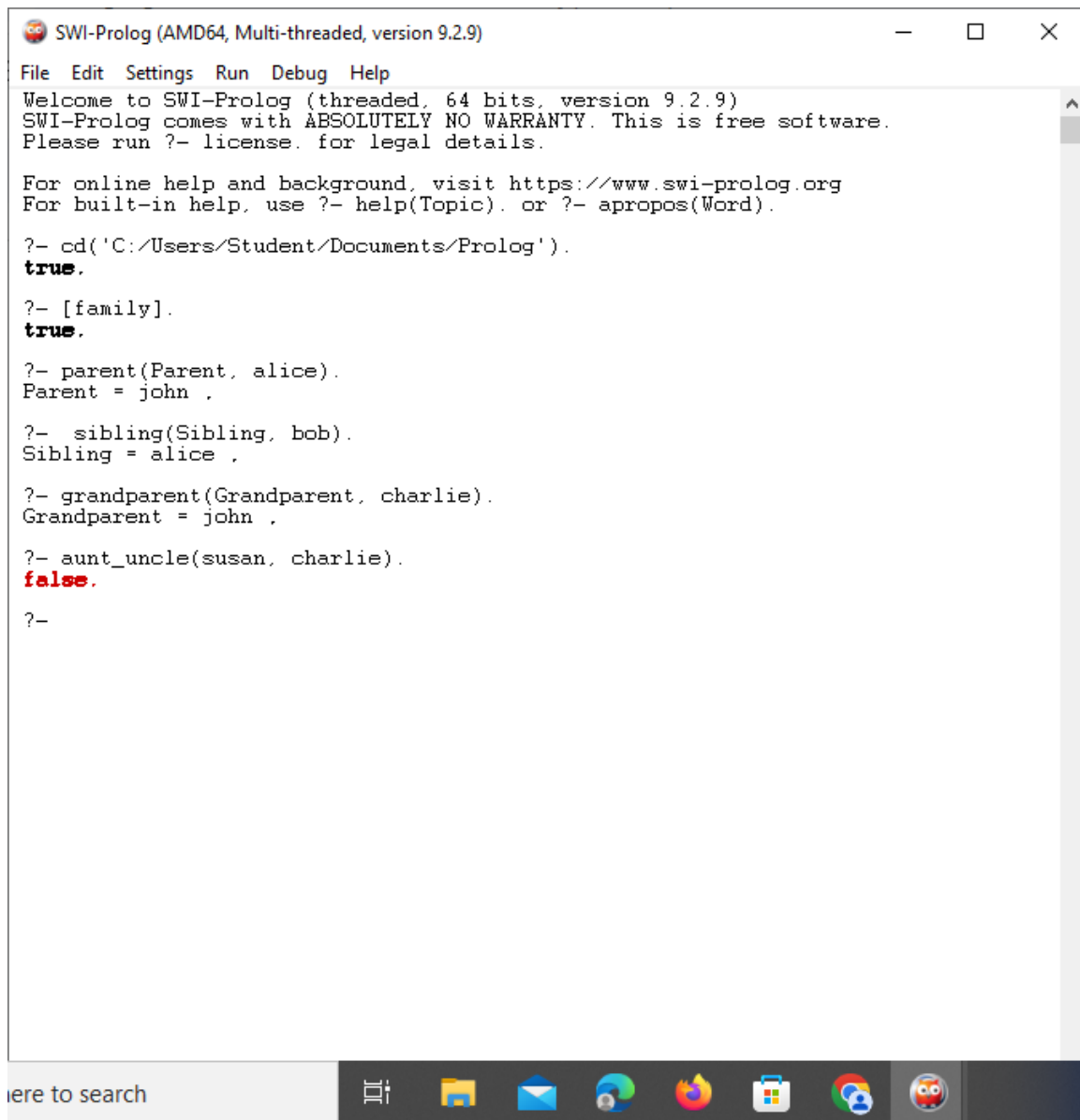
% Rule to determine siblings (X and Y are siblings if they share at least one parent)
sibling(X, Y) :-
    parent(P, X), parent(P, Y), % X and Y share at least one parent P
    X \= Y. % Ensure X is not the same as Y

% Rule to determine grandparents (GP is a grandparent of X if GP is a parent of X's parent)
grandparent(GP, X) :-
    parent(P, X), % P is a parent of X
    parent(GP, P). % GP is a parent of P (X's grandparent)

% Rule to determine if someone is an aunt/uncle (AU is an aunt/uncle of X if AU is a sibling of X's parent)
aunt_uncle(AU, X) :-
    parent(P, X), % P is a parent of X
    sibling(AU, P). % AU is a sibling of P (X's parent)

% Rule to find the children of a particular parent
children(Parent, Child) :-
    parent(Parent, Child). % Child is a child of Parent

comment(line)
Line: 1
```



```
SWI-Prolog (AMD64, Multi-threaded, version 9.2.9)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- cd('C:/Users/Student/Documents/Prolog').
true.

?- [family].
true.

?- parent(Parent, alice).
Parent = john ,

?- sibling(Sibling, bob).
Sibling = alice ,

?- grandparent(Grandparent, charlie).
Grandparent = john ,

?- aunt_uncle(susan, charlie).
false.

?-
```

B.3 Observations and learning:

- ◆ **Logical Rule-Based Programming** – PROLOG allows defining facts and rules to infer relationships dynamically.
- ◆ **Declarative Approach** – No loops or conditions are required; PROLOG automatically finds solutions using **unification and backtracking**.
- ◆ **Efficient for AI Applications** – Used in expert systems, natural language processing, and decision-making systems.

B.4 Conclusion:

In this case study, we implemented a simple family tree using Prolog. The program defines basic family relationships such as parent-child, siblings, grandparents, and aunts/uncles. Prolog's logic-based approach allows for easy querying of these relationships, which makes it an excellent tool for tasks like this.

This simple family tree could be extended further by adding more complex relationships, such as cousins, marriages, or family events. Additionally, Prolog's ability to handle logical rules and queries makes it ideal for more sophisticated applications like expert systems and natural language processing.

B.5 Question of Curiosity

Q1) Consider the following bachelor Prolog program.

What would it be the "INCORRECT" result of the following query?

```
bachelor(P) :- male(P), not married(P).
```

```
male(henry).
```

```
male(tom).
```

```
married(tom).
```

A. ?- bachelor(henry).

yes

B. ?- bachelor(tom).

no

C. ?- bachelor(Who).

Who=henry

D. ?- married(X).

X=tom

E. ?- male(P).

no

Ans: E

Q2) Which of the following is *not* a query? (I.e., which of the following does not conform to the syntax of queries?)

☒ ☐ ?- student(Lisa, 5).

☐ ?- student(Lisa, X), student(Abraham, X).

☐ ?- student(Abraham, X)

Ans: A

